Clase 11 · UART, control de flujo, RS-485 y One-Wire

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · lunes 5 oct 2026 · Tema 2 (2.7) · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

2.7.1 Conectores USB

Tarjeta	Conector
Raspberry Pi Pico (RP2040)	Micro-USB o USB-C según la versión
Arduino Uno y Mega	USB tipo B (el cuadrado de las impresoras)

Un cable barato que solo carga no trae los hilos de datos D+ y D−.

2.7.2 Comunicación serie asíncrona (UART)

Definición

Comunicación serie **sin alambre de reloj compartido**: cada extremo tiene su generador de reloj ajustado a los **mismos baudios**.

Trama:

1. Reposo en **ALTO** (VCC en TTL; en RS-232, +12 V).
2. **Bit de inicio:** flanco de bajada (ALTO → BAJO); dispara en el receptor un temporizador (astable tipo 555 o RC).
3. El receptor espera **0.5 T** y muestrea cada bit **en el centro** del pulso, no en las esquinas.
4. Bits de datos y, opcional, **paridad**.
5. **Bits de paro:** ALTO durante 1, 1.5 o 2 ciclos, para descargar las capacitancias parásitas de cables largos.

Paridad: 50 % de efectividad

Si cambia un bit, la cuenta de unos cambia y se detecta; si cambian **dos**, la cuenta vuelve a coincidir y no se detecta. Solo detecta errores de un número impar de bits: "es mejor que nada".

Ejercicio (N81 = 1 inicio + 8 datos + 0 paridad + 1 paro = 10 bits por byte) con módem de 56,600 baudios:

Paso	Cálculo
Rendimiento	56,600 ÷ 10 = **5,660 bytes/s** ≈ 5.66 KB/s
MP3 de 3 MB	3,000,000 ÷ 5,660 ≈ **530 s ≈ 8.8 min**
MP3 de 5 MB	5,000,000 ÷ 5,660 ≈ **883 s ≈ 14.7 min**

En la práctica tardaba de 20 min a 1 h por retardos de la línea. Error común: baudios distintos en emisor y receptor → "basura".

2.7.3 Control de flujo

	Hardware: RTS/CTS	Software: XON/XOFF
Cómo	Dos cables extra: el emisor activa **RTS** (Request to Send), el receptor activa **CTS** (Clear to Send) solo si tiene búfer libre	Bytes de control por el mismo hilo: **XOFF = Ctrl+S (ASCII 19)** pausa; **XON = Ctrl+Q (ASCII 17)** reanuda
Época	CPU muy lentas (cristales de 32 kHz) y RAM cara: sin búferes	CPU de 8–20 MHz y RAM barata para búferes por interrupciones: se quitan dos cables

Error común: dejar activado RTS/CTS con un cable de 3 hilos (TX, RX, GND): el emisor espera un CTS que nunca llega.

2.7.4 RS-485 (EIA-485)

Capa física industrial con **transmisión diferencial** sobre par trenzado y redes multipunto.

- El receptor mide la **diferencia** entre A y B (±1.5 V; modo común de −7 V a +12 V). El trenzado cancela campos magnéticos: blindaje natural ante motores de 20–40 HP.
- Half-duplex (2 hilos) o full-duplex (4); **bus encadenado (daisy-chain)** con 1 maestro y hasta 32 transceptores. Error común: armarla en estrella (reflexiones).

Parámetro	RS-232	RS-422	RS-485
Señal	No diferencial (a GND)	Diferencial	Diferencial
Receptores	1 (punto a punto)	10 (multidrop)	32 (multipunto)
Modo	Full duplex	Full / half	Half / full
Distancia	15 m	1.2 km	1.2 km (a 100 kbps)
Velocidad	20 kbps (115.2 kbps corto)	10 Mbps	35 Mbps (≈ 12 m)
Voltajes	±5 a ±12 V	±1.5 V	±1.5 V (−7 a +12 V)

2.7.5 One-Wire y alimentación parásita

Protocolo serie asíncrono de Dallas Semiconductor (Maxim): **datos y energía por un solo hilo** más GND.

- Línea de datos a VCC (3.3 o 5 V) con **pull-up de 4.7 kΩ**.

> **Alimentación parásita**
>
> Cada sensor tiene un **diodo** y un **capacitor de 800 pF**. Con la línea en ALTO, el capacitor se carga a través del diodo; en BAJO, el diodo bloquea la descarga y los 800 pF alimentan la lógica. En modo parásito puro, **VDD y GND del sensor se cortocircuitan a tierra**.
>
> - Velocidad baja (2–3 kbps); ≈ 100 m en modo parásito, ≈ 300 m con cable blindado y transceptor. Ej.: sensores DS18B20 en ductos, hornos o invernaderos.

2.7.6 Búsqueda de direcciones ROM de 64 bits

Campo	Bits
Código de familia (0x28 = DS18B20)	8
Número de serie grabado con láser (2^{48} ≈ 281 billones)	48
CRC	8

1. **Reset y presencia:** el maestro pone la línea en BAJO **480 µs** (8 bits a 9,600 baudios), vaciando los capacitores; tras 15–60 µs los esclavos jalan a BAJO 60–240 µs ("aquí estoy").
2. **Search ROM (0xF0):** cada esclavo transmite un bit de su dirección y luego su **complemento**; el maestro lee dos veces:

Lectura	Significado

0 1	Todos tienen 0 en esa posición
1 0	Todos tienen 1
0 0	**Colisión:** hay sensores con 0 y con 1
1 1	Falla: nadie respondió

1. Ante colisión el maestro elige una rama (escribe 0 o 1); los que no coinciden pasan a reposo.
2. **64 iteraciones** por dispositivo; luego se exploran las ramas no tomadas hasta listar todos.

2.7.7 One-Wire sobre UART y en Linux

- **Reset:** UART a **9,600 baudios** enviando **0xF0**. **Bits:** UART a **115,200**: 0xFF escribe un '1' o lee; 0x00 escribe un '0'. TX y RX se unen al hilo con una resistencia de 10 kΩ y un diodo.
- **Linux (Raspberry Pi):** módulo `w1-gpio`; cada sensor aparece en `/sys/bus/w1/devices/` como carpeta con su ROM (`28-00000xxxx`). `cat /sys/bus/w1/devices/28-xxxxx/w1_slave` muestra el CRC y `t=23125` = **23.125 °C** (miligrados).
- Error común: generar los tiempos desde C en espacio de usuario; el scheduler arruina la temporización.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

UART. Sin reloj compartido; inicio = flanco de bajada; muestreo a 0.5 T (centro del bit); paro 1/1.5/2. Paridad: solo errores impares (50 %). N81 = 10 bits por byte: 56,600 baudios → 5,660 B/s; 3 MB ≈ 8.8 min.

Control de flujo	Para recordar
RTS/CTS	2 hilos extra; época de CPU lenta y RAM cara
XON/XOFF	Ctrl+Q (17) reanuda / Ctrl+S (19) pausa; por el mismo hilo

RS-485. Diferencial, par trenzado (inmune a motores), bus encadenado, 32 nodos, 1.2 km. RS-232: 15 m, punto a punto, referido a GND.

One-Wire (examen)

Un hilo + GND, pull-up 4.7 kΩ. Parásita: diodo + 800 pF, VDD y GND a tierra. ROM = 8 familia + 48 serie + 8 CRC. Reset 480 µs; búsqueda bit + complemento: 0 0 = colisión. UART: 9,600 (reset 0xF0) y 115,200 (bits). Linux: `/sys/bus/w1/devices/…/w1_slave`, t en miligrados.